

VC

2026年 ・ 非エンジニア向けシリーズ

VibeCODE セミナー。

非エンジニアのための AI × バイブコーディング

01

第1部

AIと バイブコーディング の 理論。

AIの本質を理解し、バイブコーディングがすべてを変える理由を学ぶ。

A

第1部 ・ セクション1

人工知能の 謎を解く。

誇張なし、専門用語なし。AIの正体をわかりやすく語る。

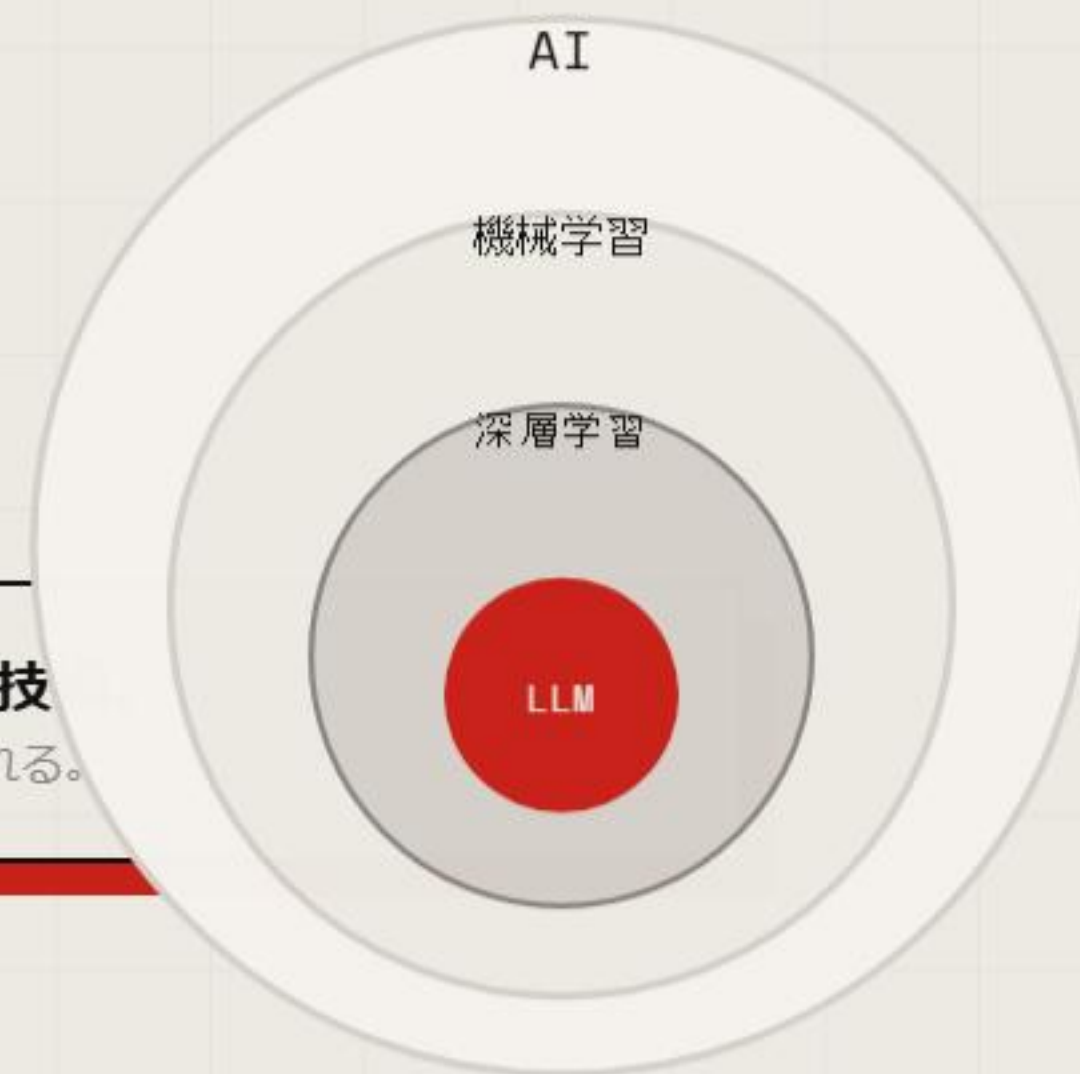
04

AIとは、本当に何か？

大きな傘。

AI

通常は人間の知性を要するタスクをこなすコンピューター技
パターン認識、意思決定、言語理解。他のすべての技術はこの円の中に含まれる。



AIとは、本当に何か？

AIの系譜

最上位 ・ 01

人工知能

サブセット ・ 02

機械学習

サブセット ・ 03

深層学習

コア ・ 04

LLM・NLP

AI

通常は人間の知性を要するタスクをこなすコンピューター技術。
最上位の概念。以下の技術はすべてここに含まれる。

ML

明示的にプログラムするのではなく、データからパターンを学習する。
例：ユーザーの行動からスパムを学習するメールフィルター。

DL

脳の仕組みに着想を得たニューラルネットワーク。複雑なパターンに強い。
例：スマホの顔認証、自動運転車の視覚システム。

NLP

テキストを読み・理解し・生成する。コンピューターと人間の言語をつなぐ。
例：Siriのような音声アシスタント、リアルタイム翻訳アプリ。

人工知能 - 詳しく見る

最上位の概念。

パターン認識・意思決定・言語理解など、通常は人間の知性が必要なタスクをこなすコンピュータ技術。

ML系の技術群

今回の焦点

機械学習 → 深層学習 → NLP → LLM。
このセミナーで追うAIの系譜であり、
ChatGPTやCopilotを動かす技術の根幹。

その他の分野

ML以外

ロボティクス（動作計画・運動学・熱制御など）は物理学・機械工学と交差する。今日のロボット工学もMLを活用する場面が増えている。

現代の現実

MLが主流

現在のAIのブレークスルーはほぼMLが牽引している。以前は、手動でコーディングした「もし〜なら」のルールベースシステムが主流だった。

ML

教師あり、 教師なし & 自己教師あり

タイプ ・ 01

教師あり学習

ラベル付きデータから学習する。「ラベル」とは人間のアノテーション。例：猫を囲むバウンディングボックスを描き、モデルに猫を認識させる。



タイプ ・ 02

教師なし学習

ラベルなしでパターンを発見する。モデルが類似性でデータをグループ化する。人間が「何を探すか」を教える必要はない。

例：購買行動による顧客クラスタリング。

タイプ ・ 03 ・ LLM

自己教師あり学習

生データから自ら学習タスクを作成する。ラベル不要。例：文中の単語を隠し、モデルに穴埋めさせる。データ自体が教師となる。

機械学習 - 10年間の変革

古典的手法 vs 深層学習

2012年以前

手作業による特徴設計

線形回帰・決定木・k-NN・SVM。

専門家が手動で「特徴量」を選ぶ。少量の構造化データに強い。

古典的ML

線形回帰

決定木

k-NN ・ SVM

2012

AlexNet

深層学習

ニューラルネットワーク

特徴を自動学習

データで拡張可能

→ LLMの動力源

2012年～現在

特徴を自ら学習

特徴エンジニアリング不要 — 生データから重要な特徴を自動で発見。

現代のAIの主流。すべてのLLMの動力源。

深層学習 - ニューラルネットワーク

ニューラル ネットワーク

脳のニューロン構造にヒントを得た、層状に接続されたノード群。各ノードはネットワークが学習した有用な情報の断片を表す。

- 入力層 - 生データを受け取る
- 隠れ層 - 重要なパターンを学習する
- 出力層 - 予測または答えを出力する

「深い (Deep)」 = 隠れ層が多い。LLMはトランスフォーマー型ネットワーク—— 数十億のパラメーターで次の単語を予測するよう学習されている。



NLP

自然言語処理 - 詳しく見る

言語と機械の出会い。

人間の言語を読み・理解し・生成する能力をコンピューターに与えるAIの分野。自然な言葉と機械の論理をつなぐ橋。

活用場面

応用例

検索エンジン・リアルタイム翻訳・チャットボット & バーチャルアシスタント・ブランドモニタリングのための感情分析

身近な例

具体例

「近くのおすすめピザ」を理解する
Google・Google翻訳・質問に答えるSiri
・コードを書いて解説するChatGPT

系譜の中での位置

橋渡し役

NLP + 深層学習 = 大規模言語モデルの基盤。AIを会話的に感じさせる。NLPなしにChatGPTは存在しない。

大規模言語モデル

チャットを動かすエンジン。

パラメーター数の規模

小規模

~100万パラメーター

中規模

~10億パラメーター

LLM

1000億以上のパラメーター

数十億の単語で学習。次に来る最も確率の高い単語を予測する。それだけ。

深層学習 ・ NLP ・ コア技術

GPT-4 ~1.8兆 ・ LLaMA 3 ~700億 ・ Gemini Ultra ~1兆以上

なぜ「大規模」？

インターネットのほぼ全体が学習データ。

数十億のニューラル「パラメーター」がパターンを記録する。データ多+パラメーター多=高性能モデル。

07

LLMの思考の仕組み

次の単語の
確率。

入力：「学習の最善策は...」



文脈+確率→人間らしいテキスト。1回の返答で数十億回繰り返される。

VC

第1部 ・ セクション2

現代の開発 & バイブ コーディング。

今日のソフトウェア開発の実態、そしてあなたの新たな役割。



ソフトウェア開発ライフサイクル

5つのフェーズ。

フェーズ 01

計画

何を作るか、なぜ作るかを決める。

フェーズ 02

設計

画面とフローをワイヤーフレームにする。

フェーズ 03

開発

現実にするコードを書く。

フェーズ 04

テスト

壊して、直して、動くことを証明する。

フェーズ 05

リリース

使う人々のもとへ届ける。

バンプコーディングはこれらのフェーズを変えない。変わるのは、第3フェーズを誰が担当するか、だ。

ソフトウェア開発手法

ウォーターフォール vs アジャイル.

従来型 ・ ウォーターフォール

順序固定 ・ 変更困難

最初に全計画を立てる。各フェーズは前の完了後に開始。途中の変更は高コスト。

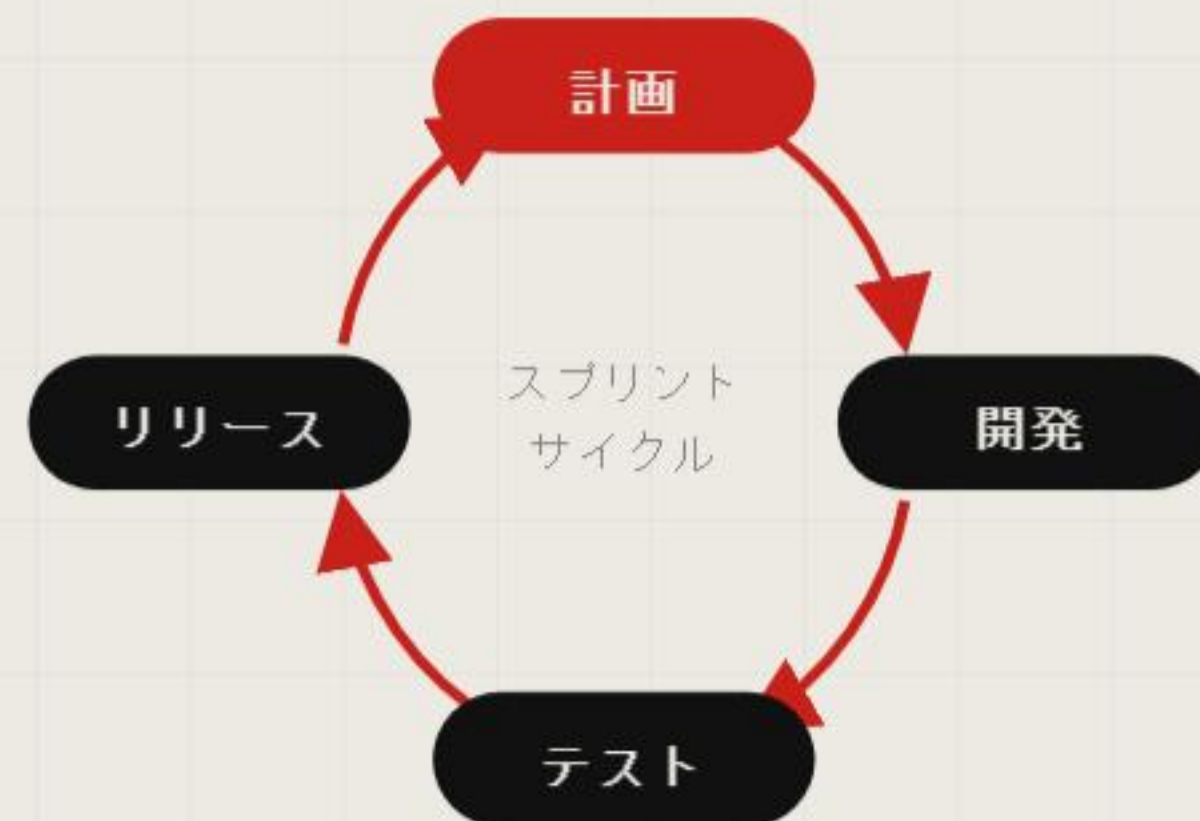


← フェーズが終わると後戻りできない

現代型 ・ アジャイル / スクラム

反復型 ・ 柔軟

短いスプリントで作業。変化に素早く対応。段階的に開発し、早期テスト・頻繁な改善を繰り返す。



11

バイブコーディングとは？

説明する。
AIが作る。

やりたいことを日常の言葉で伝える。AIがコードを書く。あなたの役割は、指示し、テストし、改善すること。

GITHUB COPILOT AGENT ・ 自然言語 → コード

なぜ
「VIBE」？**構文ではなく、意図でコーディングする。**

括弧やセミコロンに悩む必要はない。欲しいものの「雰囲気（バイブ）」を伝えるだけ。あなたの役割はコードを打つことから、AIに指示を出すことへと変わる。

12

プロジェクトの始め方、2つの方法

賢く選ぼう。

方法A

まずコード、後で調整

すぐ始めて速く作る。でも仕様がなければAIはブレる。作るよりも修正に時間を取られる。

方法B ・ 推奨

コードの前に探索する

まず要件を定義する。AIが意図通りに作る。手戻りが少なく、成果も高い。

13

仕様駆動開発

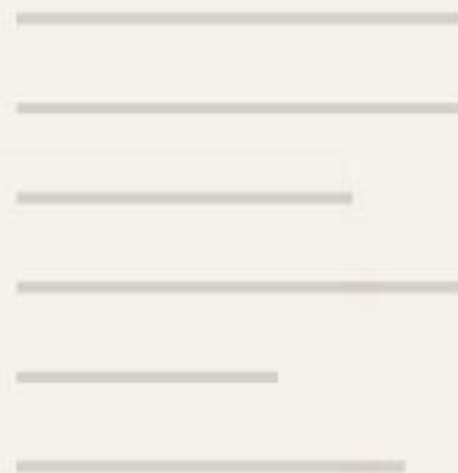
設計図を作ろう。

PRD

プロダクト要件定義書 — AIへの設計図。

目的・ユーザー・機能・ページ・データ・制約。
設計図が明確なほど、AIは高品質なものを作る。

PRD



SDD

仕様駆動開発 - 協働ループ

人間 ↔ エージェント の流れ。



すべてのフェーズで人間が関与する。AIが草案を作り、あなたが判断する。

TDD

テスト駆動開発 - 品質ループ

まずテスト、 自信を持ってリリース。



ユニットテスト・統合テスト・E2Eテスト。品質はあとから付け足すものではなく、最初から組み込むもの。

16

バイブコーディングのベストプラクティス

3つのルール。

- 01 具体的に指示する。曖昧なプロンプトは的外れなアプリを生む。
- 02 モデルをタスクに合わせる。計画にはOPUS、実行にはSONNET。
- 03 繰り返す。レビュー、テスト、改善、そしてまた繰り返す。



ベストプラクティス - 適切なモデルを選ぶ

適材 適所。

計画フェーズ

考えて戦略を立てる

深い推論が必要なタスクにはOpusかGPT-5.5を使う。仕様の作成、アーキテクチャの設計、コードを書く前の複雑な問題の分解に適している。

実行フェーズ ・ エージェントモード

作って生成する

速くて確実なコード生成にはSonnetかCodexを使う。複数ファイルのエージェントタスク、リファクタリング、素早い機能実装に向いている。

同じ問いでも、役割が違う。考えるモデルで計画し、動くモデルで実行する。

02

第2部

実践。

ツール、技術、そしてステップごとのウォークスルー。実際のアプリを作る。

T

第2部 ・ セクション1

ツール &
技術.

3つのもの。それだけで動くウェブアプリが作れる。

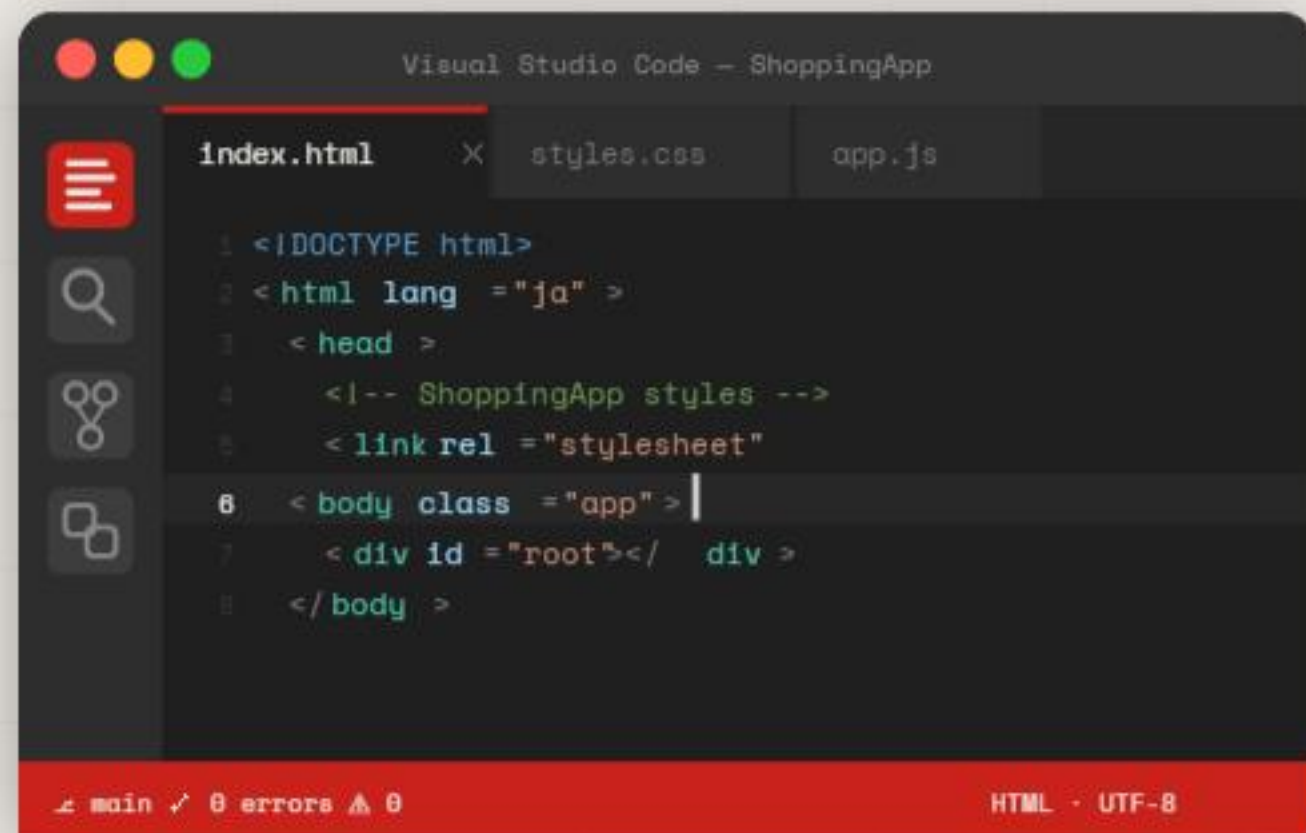


あなたの作業環境

Visual Studio Code.

プロジェクトが生きるコードエディター。無料。どのPCでも動く。中にGitHub Copilotが組み込まれている。

VSCODE ・ 無料 ・ [CODE.VISUALSTUDIO.COM](https://code.visualstudio.com)



AIペアプログラマー

GitHub Copilot.

VSCoideに直接組み込まれたAIペアプログラマー
OpenAIとMicrosoftが提供。単なる行補完
プロジェクト全体を理解する。

GITHUB ・ サブスクリプション ・ GITHUB.COM/COPILOT



なぜ
「COPILOT」?

副操縦士は飛行機を操縦するのではなく、パイロットをサポートする。

あなたが主導権を持つ。Copilotは定型コードを処理し、パターンを提案し、複数ファイルにまたがるタスクをこなし、監視を失わずに10倍速で作業を進める手助けをする。

20

GITHUB COPILOT - AIアシスタント

3つのモード.

モード ・ 01

質問 (ASK)

コードについての質問。理解。提案。概念の学習。

モード ・ 02

編集 (EDIT)

コードのリファクタリング。新機能の追加。既存ファイルの改善。

モード ・ 03 ・ メイン

エージェント (AGENT)

自律的な複数ステップのタスク。目標を与えれば、ステップを自力で進める。

21

WEBの構成要素

3つのファイル。

HTML

ファイル・01

骨格。

ページのすべての骨格 — 見出し、ボタン、フォーム、カート、商品グリッド。ブラウザに「何が、どこに存在するか」を伝える。

```
// index.html
<main>
  <h1>ショップ</h1>
  <section id="cart">...</section>
</main>
```

CSS

ファイル・02

服と化粧。

色・フォント・間隔・レイアウト、そしてスマホでの表示方法。骨格を「使いたいもの」に変える。

```
/* styles.css */
.card {
  padding: 24px;
  border: 2px solid #0E0E0E;
}
```

JS

ファイル・03

筋肉と脳。

クリックで動くもの。カートの計算、ページ遷移、商品フィルター、決済ロジック — すべての動く部品。

```
// app.js
function addToCart(id) {
  cart.push(id);
  renderCart();
}
```

TW

第2部 ・ セクション2

ステップごとの チュートリアル ウォークスルー。

13のステップ。GitHub Copilotを使って本物のショッピングアプリを作る。さあ始めよう。

01

01

ワークスペース フォルダを作成する

1. ファイル
2. ワークスペースにフォルダを追加
3. 名前を「VIBECODING-PROTOTYPEAPP」に設定
4. 追加

このフォルダはAIエージェントのホームになる。既存のgitリポジトリの外に置くこと。

02

02-03

COPILOTでPRDを
作成する

1. 質問モード
2. GPT-4.1
3. PRDに必要なセクションを尋ねる
4. アプリの詳細を入力
5. エージェントモード
6. VIBECODINGPRD.MDとして保存

開発を始める前にAIをブリーフィングする。明確なブリーフ=より良いアプリ。

04

04

ワイヤーフレーム を生成する

1. エージェントモード
2. PRDに基づくローファイワイヤーフレームを依頼
3. .TXTファイルとして保存（ページごとに1ファイル）

テキストベースのUIスケッチ。見た目ではなく、構造が大事。AIは何を作るかを正確に把握できる。

05

05

SHOPPINGAPPフォルダを作成する

1. エクスプローラーパネル
2. VIBECODING-PROTOTYPEAPP内に新しいフォルダ
3. 名前を「SHOPPINGAPP」に設定
4. 空のままにしておく

エージェント専用の空スペース。計画ファイルとアプリファイルを分けておく。

06

06

ファイルをチャット コンテキストに追加

1. コンテキスト追加ボタン

2. PRD + 5つのワイヤーフレーム.TXTファイル + SHOPPINGAPPフォルダを選択

開始前にAIにメモを渡す。これをしないと、AIはあなたの要件を知らない。

07

07

エージェントで アプリを開発する

1. エージェントモード
2. プロンプト：「PRDとワイヤーフレームを使ってSHOPPINGAPPフォルダにプロトタイプショッピングアプリを作成してください」
3. 監視する
4. 求められたらRUNをクリック
5. 保持

エージェントがindex.html、styles.css、app.js、copilot-instructions.mdを自律的に生成する。

08

08

アプリを実行して テストする

1. INDEX.HTMLを開く
2. 実行
3. デバッグなしで実行
4. ブラウザでテスト

- ✓ 絵文字と価格付きの10種類の果物商品閲覧できるか
- ✓ 商品詳細を表示・カートに追加できるか
- ✓ 左のナビバーを使って4つのページを移動できるか
- ✓ ウィンドウのサイズを変更 → レイアウトは適応するか？

09

09

アプリを繰り返し 改善する

1. 自分でアプリを使う
2. 改善点を見つける
3. エージェントモードで具体的なプロンプトを書く
4. 結果を確認
5. ブラウザでテスト
6. 繰り返す

プロンプトが具体的なほど、修正の精度が上がる。例：「2つのボタンをもっと離して」。1回のリクエストで複数の改善をまとめることもできる。

10

10

ワークスペースを 保存する

1. ファイル
2. ワークスペースに名前を付けて保存
3. プロジェクトフォルダ内に「VIBECODING-PROTOTYPEAPP.CODE-WORKSPACE」として保存

同じフォルダ構造と設定で再度開ける。プロジェクトが持ち運び可能になる。

ワークフロー
サイクル

1

ASK / PLAN MODE

構造を計画する

Copilotに何が必要？



2

AGENT MODE

ファイルを作成する

Copilotで実行



3

ASK / PLAN MODE

改善を計画する

何を改善する？



4

AGENT MODE

実行して改善する

WP



ステップ13 ・ クラスメートと協力する

2つの役割、 1つのアプリ。

役割A ・ プロダクトマネージャー

SDD重視

プロダクト要件定義書を担当する。AIが生成するすべての仕様と計画をレビューする。何を作り、何を变えるかを決める。

役割B ・ QAテスター

TDD重視

品質を担当する。アプリのバグを探す。AIが診断・修正するための的確なプロンプトを書く。実際に動くことを確認する。

一人が仕様を定め、一人がバグを見つける。二人で、動くものを届ける。

VC

まとめ

最も大切なスキル。

コードではなく、明確で具体的なプロンプトを書く。

VIBECODE セミナー ・ AI × バイブコーディング ・ 2026年